Software Testing – CO2 AT1 Scenario-Based Assessment

Name: Charuthya P

Registration number: 192421148

Q1. Online Banking System – ECP, BVA and Random Testing

1. Introduction

An online banking system may fail during fund transfers, especially when users enter amounts at the minimum or maximum limit. A good test design must check valid inputs, invalid inputs, boundaries and transaction results.

2. Equivalence Class Partitioning (ECP)

ECP divides inputs into groups expected to behave similarly. If the permitted amount is assumed to be ₹100 to ₹50,000, valid amounts form one class, while amounts below ₹100, above ₹50,000, zero and negative amounts form invalid classes. One or more representative values are selected from each class.

3. Boundary Value Analysis (BVA)

BVA tests the values immediately below, at and above each limit because comparison mistakes commonly occur at boundaries.

Q1: Boundary Value Test Cases

Test	Amount	Expected result
1	₹99	Reject: below minimum
2	₹100	Accept
3	₹101	Accept
4	₹49,999	Accept
5	₹50,000	Accept
6	₹50,001	Reject: above maximum

4. Evaluation of random testing

Random testing may reveal unexpected inputs, but it can miss important boundary values and repeat unhelpful tests. It should be retained as a supplementary method, not the main test strategy.

5. Improved strategy

1. Use ECP to cover valid and invalid amount classes.
2. Use BVA around minimum and maximum transfer limits.
3. Test insufficient balance, invalid account details, duplicate requests and network failures.
4. Confirm a failed transfer does not deduct money.
5. Confirm a successful transfer updates both accounts correctly and produces a transaction record.
6. Automate boundary and regression tests; use random testing for additional unexpected inputs.

6. Conclusion

Combining ECP, BVA, functional tests and supplementary random testing improves defect detection and makes fund transfers more reliable and secure.

Q2. Healthcare Application – White-Box Testing and Coverage

1. Introduction

Critical defects after deployment may indicate that the application was tested with insufficient coverage. White-box testing examines internal statements, branches and execution paths.

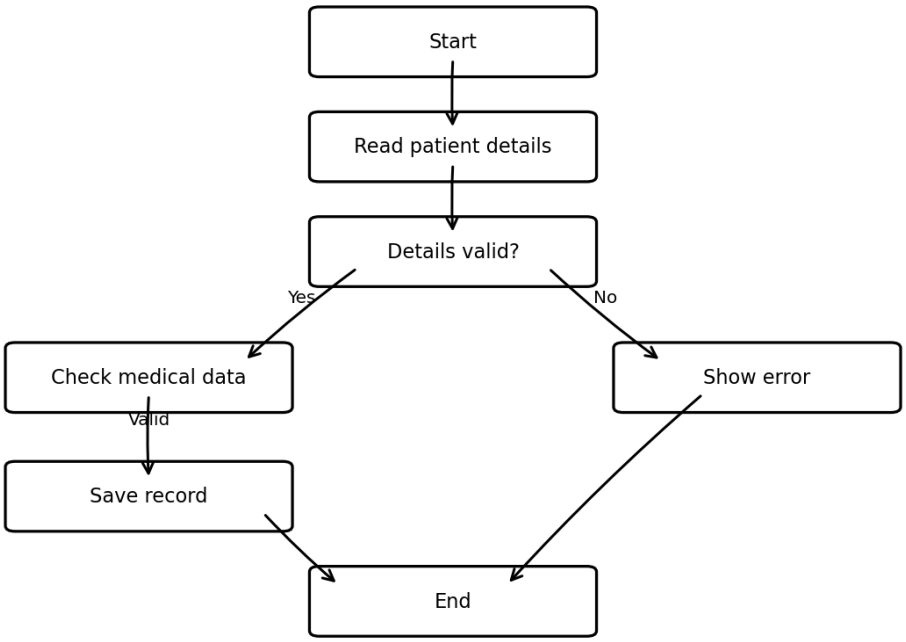
2. Test adequacy criteria

- Statement coverage: every executable statement runs at least once.
- Branch coverage: every decision outcome, such as true and false, is executed.
- Path coverage: different execution routes through the program are tested.
- Statement coverage alone is not enough because an untested branch may contain a defect.

3. Conceptual control flow graph

The following simplified example validates patient details before checking and saving medical data. The actual CFG should be built from the application code.

Q2: Conceptual Control Flow Graph



4. Missing paths

7. Valid details and valid medical data.
8. Invalid details that produce an error.
9. Valid details but invalid medical data.
10. Missing fields, boundary values and database failures.

5. Improved testing strategy

11. Execute every statement.

12. Test true and false outcomes for every decision.
13. Identify and test independent paths in the CFG.
14. Test invalid, missing and boundary data.
15. Test exception handling and database failures.
16. Use coverage tools to locate untested code and prioritize safety-critical functions.

6. Conclusion

Statement, branch and path coverage together reveal more hidden logic defects, improving patient-data accuracy, reliability and safety.

Q3. Retail E-Commerce – ECP, BVA and Decision Table Testing

1. Introduction

An e-commerce system calculates the total using price, quantity and discount. Testing each input alone can miss defects caused by combinations of values.

2. ECP

- Price: positive values are valid; zero and negative values are invalid.
- Quantity: positive whole numbers are valid if only whole units are sold; zero, negative and unsupported fractional quantities are invalid.
- Discount: 0% to 100% is assumed valid; values outside this range are invalid.

3. BVA

Q3: E-Commerce Input Boundaries

Input	Test values	Expected result
Price	0, 1, 2	Reject 0; accept positive prices
Quantity	0, 1, 2, 99, 100, 101	Reject 0 and 101
Discount	-1%, 0%, 1%, 99%, 100%, 101%	Reject values outside 0–100%

4. Decision table

A decision table lists combinations of conditions and the expected action. The total may be calculated as: Total = (Price × Quantity) × (1 − Discount/100).

Q3: Example Decision Rules

Price/quantity	Discount	Expected result
Valid	0%	Full price
Valid	10%	10% discount
Valid	100%	Zero total
Invalid	Any	Validation error
Valid	Outside 0–100%	Reject discount

5. Improved strategy

17. Define valid and invalid classes for each input.
18. Test the minimum and maximum boundaries.

19. Test combinations of price, quantity and discount.
20. Calculate expected totals manually and verify rounding.
21. Check invalid combinations and clear error messages.
22. Automate important combinations and regression tests.

6. Conclusion

ECP reduces redundant cases, BVA detects boundary defects, and decision tables test interacting conditions. Together they improve calculation accuracy and test completeness.

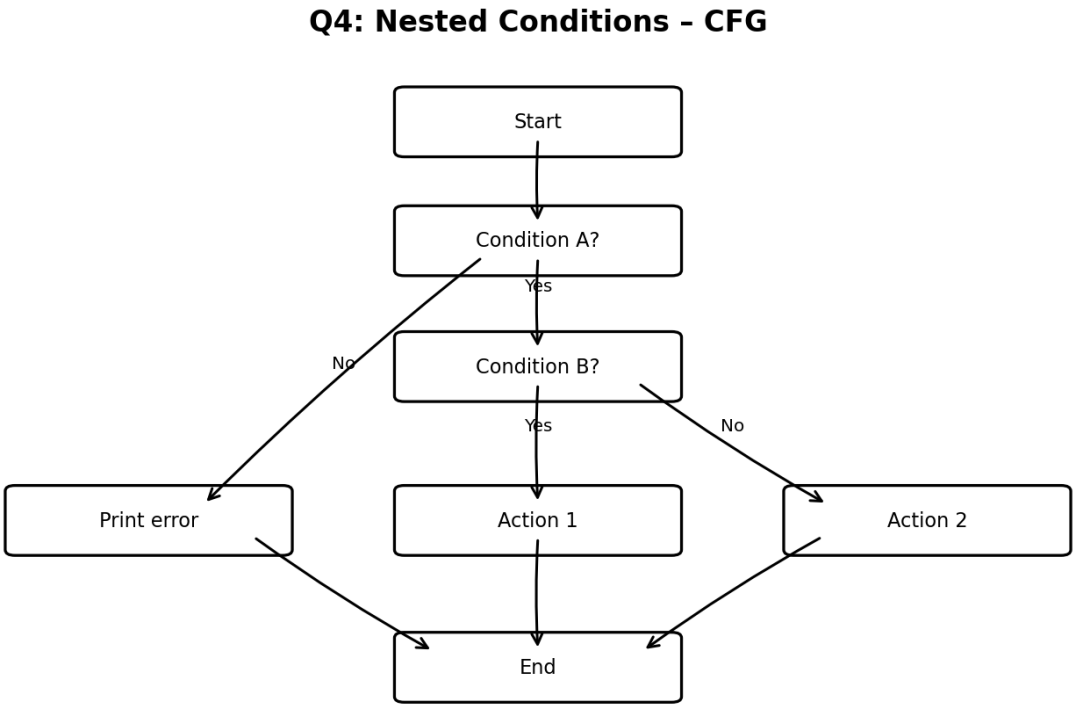
Q4. Nested Conditions and Loops – Path Testing

1. Introduction

Complex nested conditions and loops create many possible execution paths. Testing only a few paths can leave defects undetected.

2. Control flow graph

A CFG represents program execution using nodes and directed edges. This simplified example shows a nested decision.



3. Independent paths

- Path 1: Condition A is false and the error path executes.
- Path 2: Condition A is true and Condition B is true.
- Path 3: Condition A is true and Condition B is false.
- For loops, test zero, one and multiple iterations where applicable, as well as termination and boundary conditions.

4. Cyclomatic complexity

Cyclomatic complexity estimates the number of linearly independent paths: V(G) = E − N + 2P, where E is edges, N is nodes and P is connected components. For a single connected component, V(G) = E − N + 2. For a structured graph with D decision points, it is commonly D + 1.

5. Improved strategy

23. Construct an accurate CFG.
24. Identify all decision points and calculate cyclomatic complexity.
25. Design tests for independent paths.
26. Test loop entry, exit and boundary conditions.
27. Exercise nested true/false combinations.
28. Use coverage tools and investigate untested branches.

6. Conclusion

Path testing provides stronger logic coverage than statement-only testing and helps reveal incorrect conditions, non-terminating loops and unexpected behavior.

Q5. Financial Application – Hybrid Black-Box and White-Box Testing

1. Introduction

Defects may continue in a financial application when black-box and white-box tests are incomplete, coverage metrics are misunderstood or critical combinations are ignored.

2. Black-box techniques

Black-box testing checks behavior without examining internal code. ECP divides transaction inputs into valid and invalid classes. BVA checks values around the minimum and maximum transaction limits. For an assumed range of ₹100–₹50,000, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000 and ₹50,001.

3. White-box techniques

- CFG maps the program's statements and decisions.
- Path testing executes independent transaction-processing paths.
- Branch coverage checks true and false outcomes.
- Exception testing checks database errors, timeouts and failed transactions.

4. Test adequacy

Statement coverage does not guarantee that all branches or paths have been tested. Branch coverage strengthens decision testing, while basis path testing targets independent routes. No single metric guarantees that all defects are absent.

5. Improved hybrid strategy

29. Apply ECP and BVA to transaction inputs.
30. Test valid and invalid account numbers and sufficient/insufficient balances.
31. Use CFG and basis path testing for transaction logic.
32. Verify failed transfers do not deduct money and successful transfers update both accounts.
33. Test duplicate requests and concurrent transactions.
34. Automate regression tests, review uncovered code and conduct security/integration testing.

6. Conclusion

Hybrid testing covers both external input behavior and internal program logic, reducing financial errors and improving reliability.

Q6. HR Payroll – BVA and ECP

1. Introduction

Incorrect payouts at exactly 5, 10 and 15 years of service suggest mistakes in threshold comparisons, bonus ranges or combinations with performance ratings.

2. Failure analysis

The code may use > instead of >=, or ranges may overlap or leave gaps. Testing only ordinary service durations may miss exact thresholds.

3. Boundary value analysis

Q6: Service-Year Boundary Values

Boundary	Values to test	Purpose
5 years	4, 5, 6	Check lower threshold
10 years	9, 10, 11	Check middle threshold
15 years	14, 15, 16	Check upper threshold

4. Equivalence class partitioning

Divide service years into the bonus ranges specified by the actual payroll policy. Divide performance ratings into valid categories and invalid values. The assessment does not provide the bonus percentages, so expected payout amounts must come from the official policy.

5. Test cases

- 4 years with a valid rating: use the applicable lower-range bonus.
- Exactly 5 years: apply the 5-year rule.
- Exactly 10 years: apply the 10-year rule.
- Exactly 15 years: apply the 15-year rule.
- Negative service years or an invalid rating: reject as invalid.

6. Why BVA/ECP are useful

Exploratory testing depends on tester experience and can miss exact limits. BVA systematically checks thresholds; ECP reduces redundant cases by choosing representative inputs. Exploratory testing can still supplement them.

7. Improved strategy

35. Confirm official bonus rules.
36. Test immediately below, at and above every threshold.
37. Test every valid performance rating with each important service range.
38. Compare calculated bonuses with manually calculated expected values.
39. Automate regression tests after fixing the logic.

8. Conclusion

BVA and ECP systematically test payroll thresholds and input categories, helping prevent incorrect payouts.

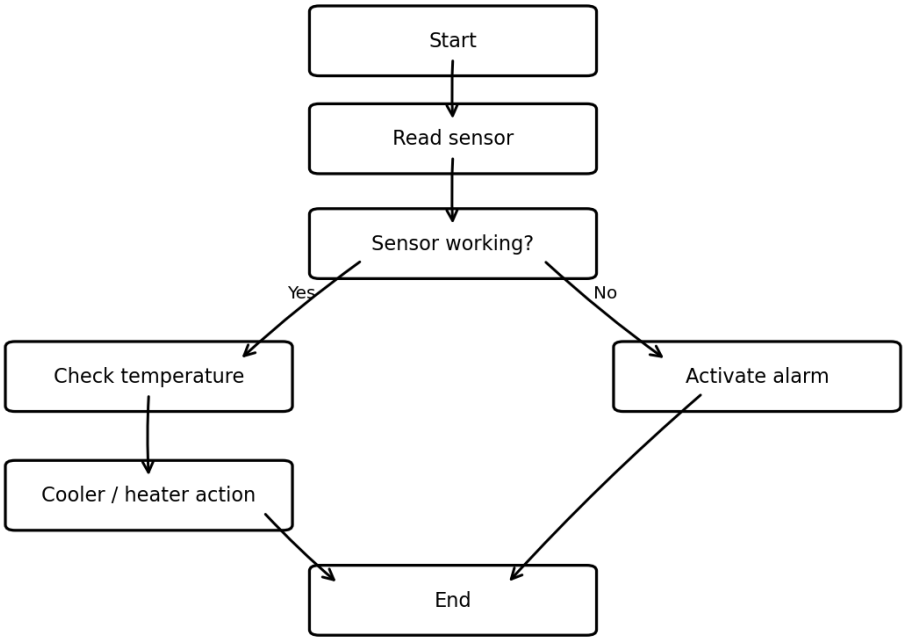
Q7. Laboratory Temperature Control – CFG and MC/DC

1. Introduction

A laboratory controller uses sensors to operate heaters, coolers and alarms. Failure to activate an alarm during sensor malfunction is safety-critical and requires structural testing.

2. Conceptual CFG

Q7: Temperature Control – Conceptual CFG



3. Branch coverage

Test both outcomes of the sensor decision: sensor working and sensor not working. Also test high, low and safe temperature ranges if supported. Testing only normal sensor readings leaves the fault branch untested.

4. MC/DC

Modified Condition/Decision Coverage requires each Boolean condition to be shown independently to affect the overall decision. For D = A AND B, where A means sensor is working and B means temperature is above the limit, use the following test pairs:

Q7: Example MC/DC Test Set

Test	A: sensor working	B: above limit	D = A AND B
T1	True	True	True
T2	False	True	False
T3	True	False	False

T1 versus T2 shows A independently changes the result while B stays true. T1 versus T3 shows B independently changes the result while A stays true.

5. Improved strategy

40. Build the CFG from actual source code.

41. Identify every Boolean condition and decision.
42. Test every branch and construct MC/DC test pairs.
43. Simulate sensor disconnection and invalid readings.
44. Verify alarm activation and safe heater/cooler behavior.
45. Test temperature boundaries and recovery after faults.
46. Record coverage and investigate uncovered conditions.

6. Conclusion

Branch testing and MC/DC provide stronger assurance than statement coverage alone and improve reliability in safety-critical control logic.

Q8. Travel Booking – Decision Table Testing

1. Introduction

A travel portal may produce a negative total when promo codes interact. Testing each code separately cannot reveal all defects caused by combinations of price, promotion type and user tier.

2. Conditions

- Ticket price is valid or invalid.
- Promotion code is valid or invalid.
- Promotion type is compatible or incompatible.
- User tier is eligible or ineligible.
- The allowed discount must not exceed the permitted total.

3. Decision table

Q8: Promotional Code Decision Table

Rule	Price valid	Promo valid	Tier eligible	Expected action
1	Yes	Yes	Yes	Calculate allowed discount
2	Yes	Yes	No	Reject / explain ineligibility
3	Yes	No	Any	Reject invalid promo
4	No	Any	Any	Reject invalid price

4. Example

If a ticket costs ₹1,000 and discounts of ₹300 and ₹800 are both subtracted, the total becomes negative. The system should enforce the business rule for combining promotions and prevent an invalid negative total.

5. Optimized strategy

47. Create rules for eligible tiers and compatible promotion types.
48. Test each distinct rule and invalid combination.
49. Test duplicate and expired codes.
50. Check discount limits and calculation order.
51. Verify total amount cannot become negative.
52. Select tests that cover each distinct rule without unnecessary duplicates.
53. Automate the selected regression cases.

6. Conclusion

Decision tables test interacting conditions and can maximize defect detection while reducing redundant tests.

Q9. Database Migration – Statement Coverage, Loops and Complexity

1. Introduction

A migration tool may show high statement coverage but still crash with infinite loops. Executing statements once does not prove that loop conditions terminate for every input.

2. Why statement coverage is insufficient

- It does not guarantee every branch executes.
- A loop may run once during testing but fail to terminate for another dataset.
- Nested loops may behave differently for empty, malformed or large datasets.
- Loop counters and exit conditions may be incorrect.

3. Path and loop testing

Model the outer and inner loops in a CFG, then test zero, one, two and many iterations where supported. Also test malformed records, maximum datasets, early exits and database errors.

4. Cyclomatic complexity

$V(G) = E - N + 2P$. For one connected component, $V(G) = E - N + 2$; for a structured graph with D decision points, it is commonly $D + 1$. This estimates independent paths, but the actual number requires the CFG or source code.

5. Improved strategy

54. Draw the CFG for nested loops.
55. Identify independent paths and loop exit conditions.
56. Test empty, single-record, multiple-record and large datasets.
57. Check counters, boundary conditions and malformed records.
58. Inject database failures and verify cleanup/recovery.
59. Use timeouts to detect potential infinite loops.
60. Review branch/path coverage and repeat regression tests after fixes.

6. Conclusion

Basis path testing, loop-boundary testing and complexity analysis provide stronger evidence than statement coverage alone and help prevent infinite-loop crashes.

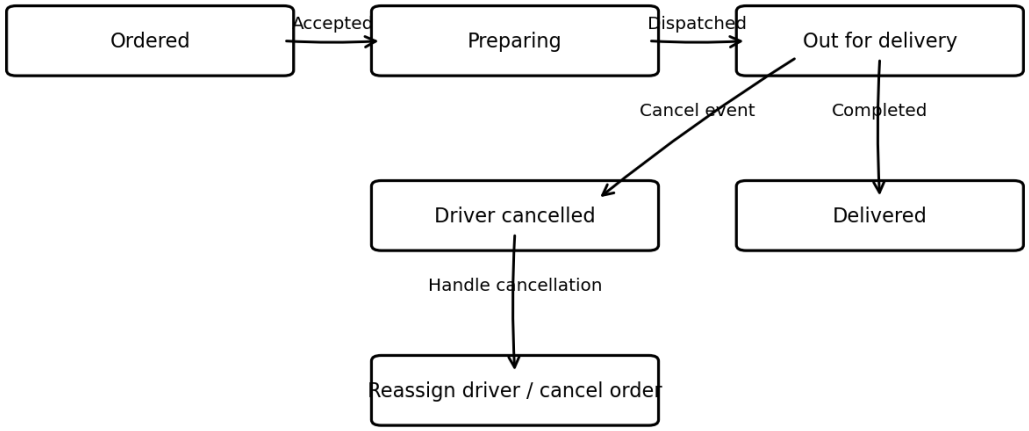
Q10. Food Delivery App – State Transition Testing

1. Introduction

The tracker moves through Ordered, Preparing, Out for Delivery and Delivered. If it gets stuck after a driver cancels, the cancellation transition may be missing or incorrectly handled.

2. State transition diagram

Q10: Food Delivery State Transition Diagram



3. Test cases

Q10: State Transition Test Cases

Initial state	Event	Expected state/action
Ordered	Restaurant accepts	Preparing
Preparing	Food dispatched	Out for Delivery
Out for Delivery	Delivery completed	Delivered
Out for Delivery	Driver cancels	Reassign driver or cancel order
Delivered	Driver cancellation	Reject invalid transition

4. Why ECP/BVA alone are insufficient

ECP groups input values and BVA checks numeric limits. They do not adequately test the sequence of events that changes an order's state. A driver cancellation must be tested in the context of the current order state.

5. Improved strategy

61. List all valid states and permitted transitions.
62. Test normal transitions and invalid transitions.
63. Test driver cancellation at different stages.
64. Verify driver reassignment, order cancellation, refund and customer notification according to policy.
65. Test network interruptions and repeated events.
66. Ensure no order remains permanently stuck.
67. Automate state-transition regression tests.

6. Conclusion

State Transition Testing verifies both the current state and the event that triggers the next state, helping prevent stuck orders and incorrect status updates.

Q11. Authentication – Error Guessing and Path Analysis

1. Introduction

A multi-factor authentication module checks passwords, OTP timing and IP location. Brute-force vulnerabilities may occur if repeated attempts are not limited or authentication checks can be bypassed.

2. Black-box testing

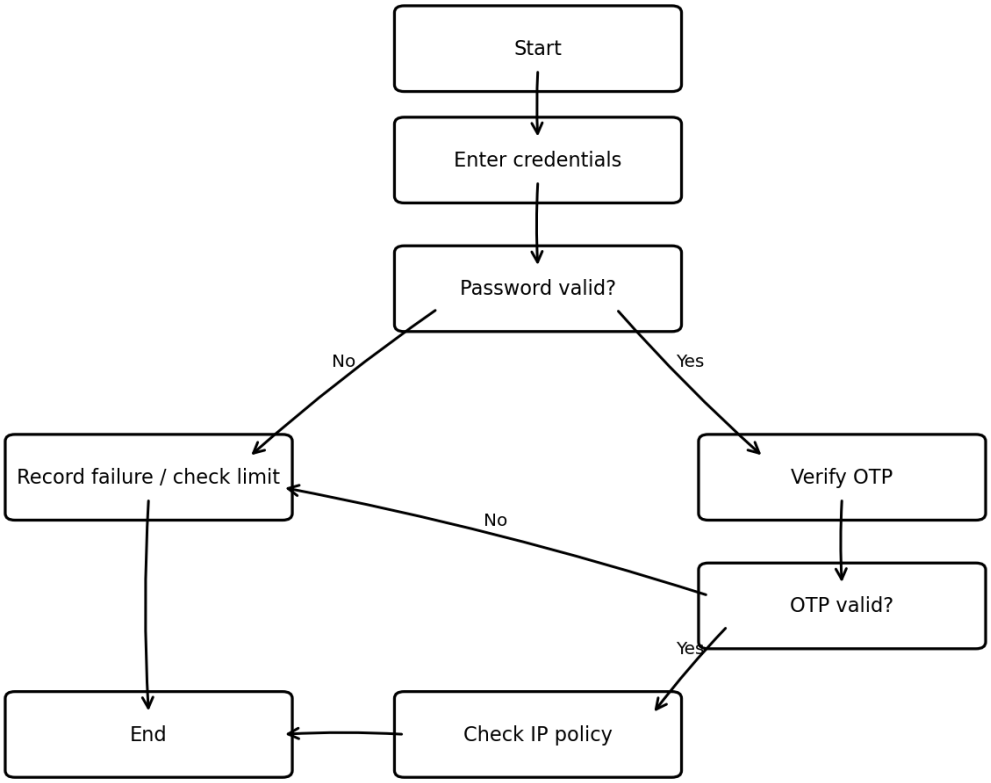
- Correct and incorrect passwords.
- Incorrect, expired and reused OTPs.
- OTP submitted after the allowed time.
- Login from unfamiliar IP locations.
- Repeated failed attempts and account lockout.

3. Error guessing

Error guessing uses experience to predict likely defects, such as unlimited attempts, missing rate limits, reusable OTPs, incorrect lockout behavior, or error messages that reveal whether an account exists.

4. Conceptual CFG

Q11: Authentication – Conceptual CFG



5. Hybrid strategy

68. Use ECP/BVA for password lengths and OTP expiry boundaries.
69. Apply error guessing to likely authentication weaknesses.
70. Use path testing for password, OTP, lockout and IP decisions.
71. Test rate limiting, account lockout and session management.
72. Verify failed authentication never grants access.
73. Review security logs without exposing secrets.
74. Use automated security tests and regression testing.

6. Conclusion

Black-box tests verify visible authentication behavior, while white-box path analysis checks internal logic. Error guessing adds practical security scenarios and improves the module's security posture.

Q12. E-Learning Subscription – ECP and Date Boundaries

1. Introduction

An e-learning platform offers Free, Basic and Premium access. Errors on the expiration date may result from incorrect date comparisons or unclear upgrade rules.

2. Equivalence classes

- Upgrade before expiration.
- Upgrade exactly on expiration date.
- Upgrade after expiration.
- Invalid subscription date.
- Upgrade from Free, Basic or Premium.
- Upgrade while active versus after expiry.

3. Date boundary tests

Q12: Expiration Date Boundaries

Test date	Relation to expiry	Purpose
24 October	One day before	Check active subscription
25 October	Expiration date	Check exact-date rule
26 October	One day after	Check expired-subscription rule

The dates above are examples. If the expiration date is 25 October, test the day before, the exact date and the day after. The system requirements must define whether expiration is inclusive or exclusive.

4. Example test cases

- Free to Basic: activate according to the subscription rules.
- Basic to Premium while active: apply upgrade rules.
- Basic expiring today: process according to the defined expiration policy.
- Basic expired yesterday: follow expired-subscription rules.
- Invalid date or unsupported subscription type: show a validation error.

5. Improved strategy

75. Define inclusive/exclusive expiration rules.
76. Test all subscription types before, on and after expiration.
77. Verify access permissions immediately after upgrade.

78. Check billing, renewal and course availability.
79. Test time zones if supported.
80. Automate date-boundary regression tests.

6. Conclusion

ECP organizes date and subscription categories, while BVA focuses on the exact expiration boundary. Together they help prevent access and billing errors.

Q13. Smart-Home Lighting – Branch and Condition Coverage

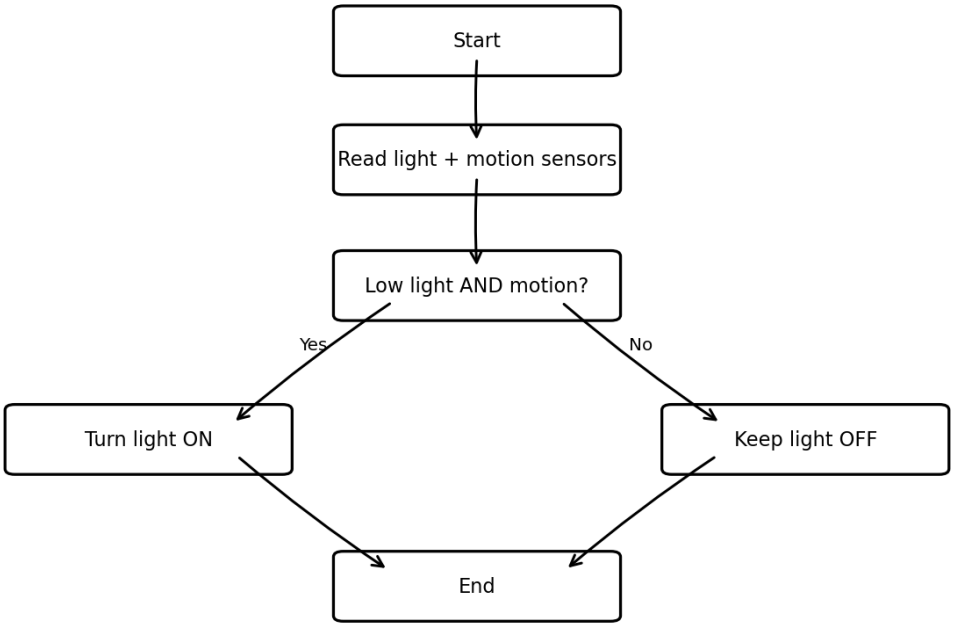
1. Introduction

A lighting controller uses sensor readings, switch-case statements and Boolean expressions. Statement coverage alone may not test every decision outcome or individual condition.

2. Example logic and CFG

Assume the light turns on only when ambient light is low AND motion is detected. The following CFG illustrates this simplified decision.

Q13: Smart Lighting – Conceptual CFG



3. Branch and condition coverage

Q13: Example Decision Tests

Test	Low light (A)	Motion (B)	Expected action
1	True	True	Light ON
2	True	False	Light OFF
3	False	True	Light OFF
4	False	False	Light OFF

Branch coverage checks true and false outcomes of the decision. Condition coverage checks that each individual Boolean condition becomes true and false. The actual tests must match the real controller's logic.

4. Improved strategy

81. Construct the CFG from source code.
82. Test all switch-case options and the default case.
83. Test true and false outcomes of each decision.
84. Test sensor values around brightness thresholds.
85. Simulate missing, faulty or disconnected sensors.
86. Verify brightness remains within allowed limits.
87. Check hardware responses against software commands.
88. Automate regression tests.

5. Conclusion

Branch and condition coverage reveal defects that statement coverage may miss and reduce the risk of software-hardware integration failures.

Q14. Logistics System – Robust Boundary Value Analysis

1. Introduction

Shipping cost depends on package weight, destination zone and delivery speed. Missing zero and negative weights in testing can allow invalid values to reach the calculation.

2. Failure analysis

The system may test only ordinary positive weights, use incorrect comparison operators, or calculate charges before validating inputs.

3. Robust BVA

Assume the permitted weight is greater than 0 kg and up to 100 kg. Actual limits must follow system requirements.

Q14: Robust BVA for Package Weight

Test weight	Expected result
-0.1 kg	Reject
0 kg	Reject
0.1 kg	Accept
99.9 kg	Accept
100 kg	Accept
100.1 kg	Reject

4. Additional negative tests

- Valid weight with invalid destination zone.
- Valid weight with unsupported delivery speed.
- Missing or non-numeric weight.
- Extremely large weight.
- Weight at the maximum permitted limit.

5. Improved strategy

89. Define actual minimum and maximum weight limits.
90. Test just below, at and above both boundaries.
91. Validate destination zone and delivery speed.
92. Test combinations of weight, zone and speed.
93. Verify shipping calculations manually.
94. Reject invalid values before calculation.
95. Check error messages and automate regression tests.

6. Conclusion

Robust BVA covers valid and invalid boundaries. Negative testing helps ensure that invalid weights are rejected and shipping charges remain accurate.

Q15. Medical Imaging – Basis Path Testing

1. Introduction

Medical imaging software uses five parallel processing loops. Inconsistent results for prime-sized datasets may indicate a problem in data partitioning, loop boundaries or synchronization.

2. Limitations of random testing

Random testing may not deliberately include prime-sized datasets or important loop boundaries. It should supplement structured tests rather than replace them.

3. White-box and basis path testing

White-box testing examines internal control flow, loops, data partitioning and synchronization. Basis path testing identifies linearly independent execution paths using a CFG.

4. Cyclomatic complexity

V(G) = E − N + 2P. For one connected component, V(G) = E − N + 2. The number of paths to test depends on the actual CFG; the question does not provide source code to calculate a specific value.

5. Dataset test cases

Q15: Dataset Size Test Cases

Test	Dataset size	Purpose
1	0 (if supported)	Empty-input handling
2	1	Minimum data
3	7 (prime)	Prime-size behavior
4	8 (composite)	Compare partitioning
5	Large prime	Parallel processing at scale
6	Large composite	Compare processing

6. Improved strategy

96. Construct the CFG and identify independent paths.
97. Test each independent path and loop termination condition.
98. Compare prime-sized datasets with composite-sized datasets.
99. Verify each element is processed exactly as intended.

100. Check parallel-task synchronization and data partitioning.
101. Compare output with a trusted reference implementation.
102. Repeat tests to detect inconsistent results.
103. Monitor memory, timing and stability for large datasets.
104. Use random tests as supplementary checks.

7. Conclusion

Basis path testing systematically examines independent paths. Carefully chosen prime and composite dataset sizes help investigate the reported issue. Numerical validation, loop testing and supplementary random tests improve accuracy and stability.